

Fase 3 — Juegos como módulos · Abril

12

Implementar Palabra Secreta

Si la arquitectura es buena, añadir un juego nuevo no rompe nada. Hoy lo demostramos.

Qué vas a hacer hoy

En la **Sesión 11** diseñaste Palabra Secreta en pseudocódigo y descubriste qué cambia y qué se queda igual respecto al Tic Tac Toe. Hoy toca **el momento decisivo**: ver si la arquitectura que construimos en febrero y marzo aguanta cuando le metemos un juego completamente distinto.

IDEA CENTRAL

**Un juego nuevo no se programa desde cero.
Se *conecta* al sistema que ya existe.**

- | | | |
|----|--------------------------------------|----------------------|
| 1. | Por qué esta sesión es diferente | — tensión |
| 2. | Conexión con la memoria del proyecto | — qué ya sabes |
| 3. | Diseño antes de código | — responsabilidades |
| 4. | Implementación guiada | — código real |
| 5. | Uso de la IA en esta sesión | — prompts y revisión |
| 6. | Mini-reto y preguntas críticas | — qué entregar |

Por qué esta sesión es diferente

Cuando hiciste Tic Tac Toe (Sesión 10) tenías una excusa: era el primer juego, así que mientras lo construías ibas **moldeando el sistema** a su medida. Eso es trampa, aunque no la viste.

Hoy no hay trampa. La arquitectura ya está. El estado central ya está. La función `navegar()` ya está. `actualizarUI()` ya está. Si todo eso lo diseñaste bien, **añadir Palabra Secreta debería ser fácil**. Si lo diseñaste regular, hoy lo notarás. Y eso, aunque incomode, es exactamente lo que queremos descubrir.

El test silencioso de la ingeniería

Una arquitectura no se valida cuando hace lo que diseñaste. Se valida cuando aguanta lo que *no* diseñaste. Por eso hoy importa más que la sesión 10.

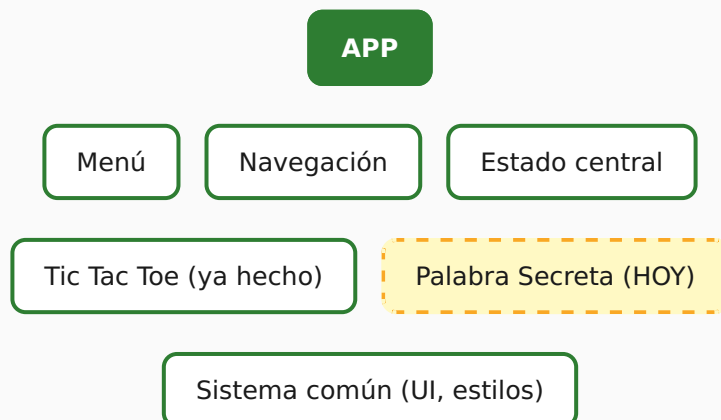
Lo que vas a aprender (sin que te lo digamos en voz alta)

- **Reutilización real:** usar piezas ya construidas sin tocarlas.
- **Aislamiento:** el código de Palabra Secreta vive en su sitio, no contamina el resto.
- **Acoplamiento bajo:** los juegos no se conocen entre sí. Ninguno sabe que el otro existe.
- **Detección de fugas de diseño:** si para meter Palabra Secreta tienes que *modificar* Tic Tac Toe, hay un problema de fondo.

Lo que ya sabes (y vas a usar tal cual)

En las sesiones anteriores construiste todo esto. Hoy **no se toca**. Solo se usa.

ARQUITECTURA ACTUAL DE LA APP (MEMORIA OPERATIVA)



El patrón que ya tienes interiorizado

evento



función



estado cambia



UI se actualiza

Este patrón funcionó para Tic Tac Toe. Hoy demostramos que también vale para la Palabra Secreta *sin cambiar ni una coma del sistema*. Eso es lo que llamamos **una abstracción que aguanta**.

Variables y funciones del proyecto que vas a tocar (y las que NO)

Pieza	¿Se toca hoy?	Por qué
estado	Sí, se añade una propiedad	Palabra Secreta necesita guardar la palabra y los intentos. Es información nueva, no modifica la existente.
navegar()	No	Ya sabe ir a cualquier <code>pantallaActual</code> . Le da igual el destino.

Pieza	¿Se toca hoy?	Por qué
actualizarUI()	Mínimo	Solo añadir el caso de la nueva pantalla. Una línea.
Tic Tac Toe	NO se toca	Si tocas algo aquí para que la Palabra Secreta funcione, has roto el aislamiento.
HTML	Sí	Hay que añadir el <code><div id="pantalla-juego-palabra-secreta"></code> .

Diseño antes de código

Antes de tocar el teclado, contesta tres preguntas. Si no sabes la respuesta, **no programes todavía**.

Pregunta 1 — ¿Qué necesita saber el estado para Palabra Secreta?

Palabra Secreta necesita guardar información que *no existe* en el resto de la app. Esa información es propia de este juego. Mírala:

```
// Lo que Palabra Secreta necesita recordar:
const palabraSecreta = {
  palabra: "jirafa",           // la palabra a adivinar
  letrasCorrectas: [],         // ['a', 'i']
  letrasIncorrectas: [],       // ['z', 'o']
  intentosMaximos: 6,
  estadoJuego: "jugando"      // 'jugando' | 'ganado' | 'sin-intentos'
}
```

Decisión de diseño importante

Esto vive **dentro** de `estado`, no como variable suelta. Como aprendiste en la Sesión 5: el estado vive en un único objeto. La regla no cambia porque el juego sea distinto.

Pregunta 2 — ¿Qué eventos provoca el usuario?

- Pulsar una letra del abecedario (clic en un botón).
- Pulsar "Reiniciar partida".
- Volver al menú principal (esto ya está: usa `navegar('menu')`).

Mira lo que ya no tienes que diseñar: la navegación. Eso es un sistema reutilizado.

Pregunta 3 — ¿Qué responsabilidad tiene cada archivo?

Archivo	Responsabilidad
<code>palabra-secreta.js</code>	Lógica del juego: validar letras, comprobar fin de partida, modificar el estado del juego.

Archivo	Responsabilidad
estado.js	Almacenar el sub-estado de Palabra Secreta dentro del estado global. <i>Sin tocar lo que ya había.</i>
app.js	Conectar eventos del HTML con las funciones del juego. Llamar a <code>actualizarUI()</code> .
index.html	Añadir la pantalla <code>pantalla-juego-palabra-secreta</code> con sus botones de letras.
styles.css	Estilos visuales. Cero lógica.

¿Por qué un archivo nuevo (palabra-secreta.js)?

Porque la lógica de Palabra Secreta no tiene nada que ver con la del Tic Tac Toe. Mezclarlas en el mismo archivo sería como meter las reglas del ajedrez y las del parchís en la misma libreta: funciona, pero es un caos cuando creces.

Implementación, paso a paso

Paso 1 — Ampliar el estado central

En `estado.js` añades una propiedad nueva. **No tocas las que ya estaban.**

```
const estado = {
  pantallaActual: "menu",
  juegoActivo: null,

  // Tic Tac Toe (ya estaba)
  ticTacToe: { /* ... */ },

  // Palabra Secreta (NUEVO)
  palabraSecreta: {
    palabra: "",
    letrasCorrectas: [],
    letrasIncorrectas: [],
    estadoJuego: "jugando"
  }
}
```

Paso 2 — Lógica del juego en su archivo

```
// palabra-secreta.js – solo lógica, nada de UI

function iniciarPalabraSecreta(palabra) {
  estado.palabraSecreta.palabra = palabra
  estado.palabraSecreta.letrasCorrectas = []
  estado.palabraSecreta.letrasIncorrectas = []
  estado.palabraSecreta.estadoJuego = "jugando"
}

function probarLetra(letra) {
  if (estado.palabraSecreta.estadoJuego !== "jugando") return

  if (estado.palabraSecreta.palabra.includes(letra)) {
    estado.palabraSecreta.letrasCorrectas.push(letra)
  } else {
    estado.palabraSecreta.letrasIncorrectas.push(letra)
  }

  comprobarFinDePartida()
}
```

```
  actualizarUI() // ← función que YA existe
}
```

Mira esto con atención

`probarLetra()` termina llamando a `actualizarUI()`, que es exactamente la misma función que usa Tic Tac Toe. **Ese es el sistema funcionando.** Palabra Secreta no necesita su propia función de actualización: usa la del sistema.

Paso 3 — Conectar al patrón del sistema

click letra "A" → `probarLetra('a')` → estado cambia → `actualizarUI()`

El mismo patrón que usaste para Tic Tac Toe. Ni una variación. Si el patrón cambia para cada juego, la abstracción está mal.

Paso 4 — Pantalla en HTML y caso en actualizarUI

```
<!-- index.html -->
<div id="pantalla-juego-palabra-secreta" class="pantalla">
  <div id="palabra-oculta"></div>
  <div id="letras-incorrectas"></div>
  <div id="teclado"></div>
  <button onclick="navegar('menu')">Menú</button>
</div>
```

Y en `actualizarUI()` añades el caso:

```
function actualizarUI() {
  ocultarTodasLasPantallas()
  mostrarPantalla(estado.pantallaActual)

  if (estado.pantallaActual === "juego-palabra-secreta") {
    renderizarPalabraSecreta() // pinta palabra, intentos, teclado
  }
  if (estado.pantallaActual === "juego-tic-tac-toe") {
    renderizarTicTacToe() // ya existía
  }
}
```

Uso de la IA en esta sesión

La IA es muy buena escribiendo lógica de juegos de adivinar palabras. La habrá hecho miles de veces. Pero la IA **no conoce tu sistema**. No ha leído tu memoria del proyecto. Si le pides "hazme un juego de Palabra Secreta", te dará uno *independiente*, no integrado en tu app. Y eso es exactamente lo que NO quieres.

Prompts buenos

- "Tengo esta función `probarLetra(letra)` que modifica `estado.palabraSecreta`. ¿Qué casos borde no estoy cubriendo?"
- "Dame dos formas de comprobar si todas las letras de la palabra están en `letrasCorrectas`, y explícame cuál es más legible."
- "Aquí está mi función `actualizarUI()`. ¿Cómo añadiría el caso de Palabra Secreta sin romper el de Tic Tac Toe?"

Prompts a evitar

- ~~"Hazme un juego de Palabra Secreta completo."~~
- ~~"Programa Palabra Secreta en mi app."~~
- ~~"Soluciona el bug, no me importa por qué."~~
- ~~"Mezcla Tic Tac Toe y Palabra Secreta en un solo archivo si funciona."~~

Qué tienes que revisar **OBLIGATORIAMENTE** en lo que te dé la IA

- ❏ ¿Modifica `estado` directamente o lo duplica en una variable suelta? **Lo duplicado es bug.**
- ❏ ¿Llama a `actualizarUI()` al final, o pinta la pantalla por su cuenta? **Si pinta, está rompiendo el patrón.**
- ❏ ¿Usa `window.location`, `alert()`, o redirige a otra página? **Antipatrón. Tu app es SPA.**
- ❏ ¿Toca algo del Tic Tac Toe? **No debería. Si lo hace, no lo aceptes.**
- ❏ ¿Mete CSS dentro del JavaScript? **Mezcla apariencia y lógica. Mal.**

El error más típico de la IA en esta sesión

Te va a generar un juego que *recarga la página* o usa `prompt()` para pedir letras. Eso ignora completamente tu sistema de eventos y de estado. **Recházalo.**

Recuerda: la IA ayuda a programar, pero no diseña el sistema. El sistema lo diseñas tú.

Mini-reto de 1 hora

Tu misión

Al final de la hora, debes poder demostrar esto:

- 📄 Desde el menú, pulsas "Palabra Secreta" → Llegas a la pantalla del juego.
- 📄 Pulsas letras y se actualiza la palabra oculta y los intentos.
- 📄 Cuando aciertas todas o se acaban los intentos, se muestra un mensaje claro.
- 📄 Vuelves al menú y desde ahí puedes entrar a Tic Tac Toe **y sigue funcionando igual que antes.**
- 📄 No has tocado ni una línea del archivo de Tic Tac Toe.

El último punto es el más importante. Si lo cumples, tu arquitectura es buena. Si no, hoy aprendes algo más valioso aún: dónde está el problema.

Preguntas para pensar (antes y después de programar)

1. Si mañana añadiéramos un tercer juego (por ejemplo, Memory), ¿cuántos archivos tendrías que crear y cuántos modificar? Si la respuesta es "creo varios y modifico pocos", vas bien.
2. ¿Qué pasaría si dos juegos quisieran usar el mismo nombre de variable, por ejemplo `turno`? ¿Tu sistema lo permite o lo rompe?
3. Si la IA te diera dos versiones de `probarLetra()` y una usa `if/else` y otra usa un objeto/diccionario, ¿cuál es mejor para esta app y por qué?
4. ¿En qué momento exacto de tu código se decide que se han agotado los intentos? ¿Está en un solo sitio o repartido?

Qué NO hacer (antipatrones de hoy)

Errores que no quieres cometer

- Copiar el archivo de Tic Tac Toe y modificarlo para hacer Palabra Secreta.

Lo que sí quieres

- Un archivo `palabra-secreta.js` con lógica pura.
- Estado del juego dentro de `estado.palabraSecreta`.

- Crear una variable global `palabraSecretaGlobal` fuera de `estado`.
- Hacer que Palabra Secreta pinte el HTML directamente sin pasar por `actualizarUI()`.
- Dejar lógica de Palabra Secreta dentro de `app.js` "porque es más rápido".
- Aceptar lo primero que te dé la IA sin leerlo.

- Cada cambio termina en `actualizarUI()`.
- El Tic Tac Toe sigue funcionando sin tocarlo.
- Saber explicar, en una frase, qué hace cada archivo.

Y la próxima sesión...

En la **Sesión 13** entramos en la **Fase 4 — Producto y calidad**. Hablaremos de bugs y testing manual: cómo pensar "qué puede fallar" antes de que falle. Hoy has demostrado que el sistema crece. La próxima vamos a demostrar que el sistema *aguanta*.

RECUERDA

**No estamos haciendo juegos.
Estamos aprendiendo a construir sistemas que
casualmente incluyen juegos.**